Homeword 3: Backward selection

Kabin Devkota

Libraries and setup

```
library(tidyverse)
library(tidymodels)
library(ggformula)
library(ISLR)
library(GGally)
library(ISLR2)
library(car)
set.seed(1998)
```

Question

Use the Auto dataset (can be found in ISLR R package).

Our objectives:

- Predict `acceleration` using the remaining quantitative variables.
- Which, if any, of the variables are significant predictors of acceleration.
- How good is the resulting prediction?

1. Remove the name and origin columns from the data set. Then produce a ggpairs plot. Write a short discussion (2–3 sentences) about what the plot suggests for multiple linear regression (collinearity, pairwise relationships, etc.).
2. Start from the full model with all quantitative predictors. Do a manual backward selection using p-values to determine what to remove until all predictors are significant at the $\alpha = 0.05$ level. Filter the tidy() output to show only candidates for removal, i.e. p-values $> \alpha$. Remove the predictor with the largest p-value at each step. Compare the full (unfiltered) tidy() results and adjusted R^2 at each step to the previous step.
3. With the final model (from (2)), check residuals, interpret coefficients, report confidence intervals on coefficients.

```
# Load the data
auto <- ISLR::Auto |> as_tibble()

# Quick look at the data
glimpse(auto)
```

```
## Rows: 392
## Columns: 9
## $ mpg          <dbl> 18, 15, 18, 16, 17, 15, 14, 14, 14, 15, 15, 14, 15, 14, 2~
## $ cylinders    <dbl> 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 4, 6, 6, 6, 4, ~
```

```
## $ displacement <dbl> 307, 350, 318, 304, 302, 429, 454, 440, 455, 390, 383, 34~
## $ horsepower   <dbl> 130, 165, 150, 150, 140, 198, 220, 215, 225, 190, 170, 16~
## $ weight       <dbl> 3504, 3693, 3436, 3433, 3449, 4341, 4354, 4312, 4425, 385~
## $ acceleration <dbl> 12.0, 11.5, 11.0, 12.0, 10.5, 10.0, 9.0, 8.5, 10.0, 8.5, ~
## $ year         <dbl> 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 7~
## $ origin       <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 3, ~
## $ name         <fct> chevrolet chevelle malibu, buick skylark 320, plymouth sa~
```

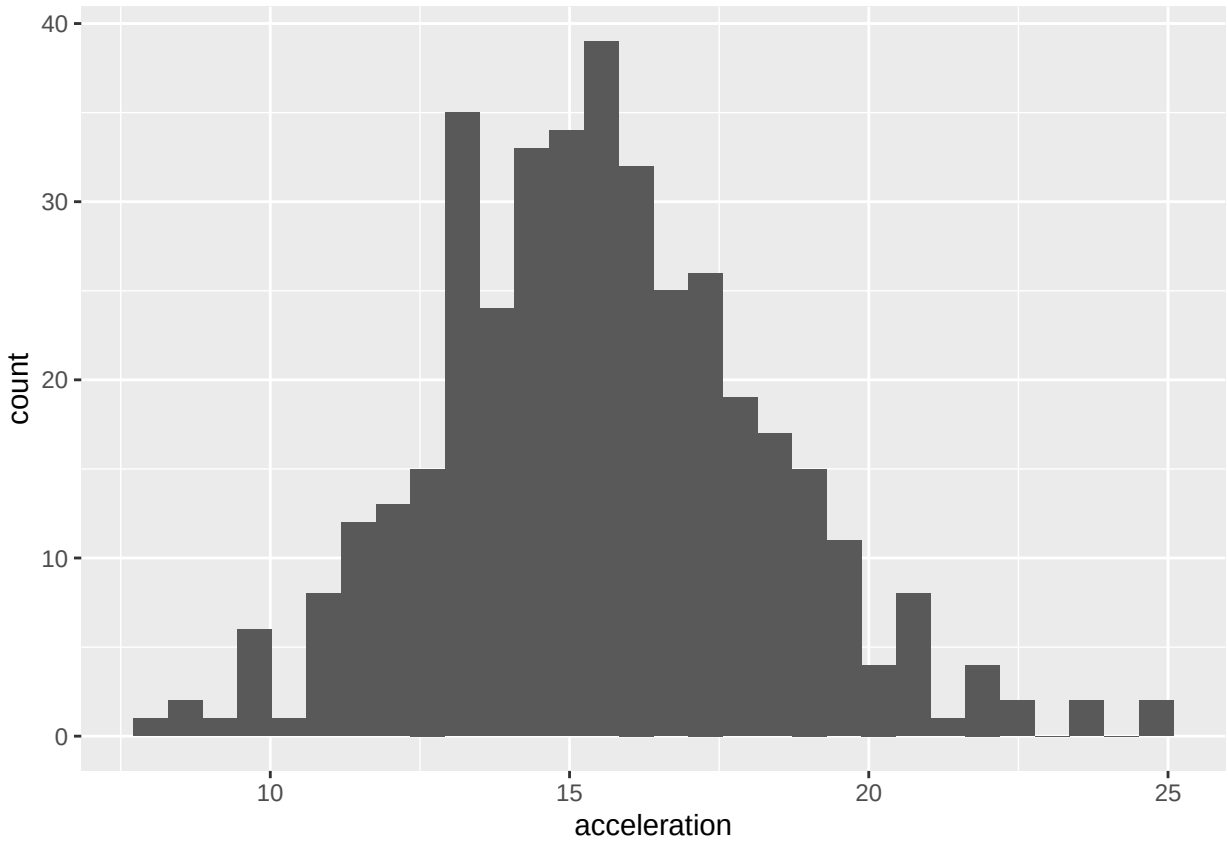
```r
summary(auto)
```

```
##       mpg          cylinders      displacement     horsepower        weight    
##  Min.   : 9.00   Min.   :3.000   Min.   : 68.0   Min.   : 46.0   Min.   :1613  
##  1st Qu.:17.00   1st Qu.:4.000   1st Qu.:105.0   1st Qu.: 75.0   1st Qu.:2225  
##  Median :22.75   Median :4.000   Median :151.0   Median : 93.5   Median :2804  
##  Mean   :23.45   Mean   :5.472   Mean   :194.4   Mean   :104.5   Mean   :2978  
##  3rd Qu.:29.00   3rd Qu.:8.000   3rd Qu.:275.8   3rd Qu.:126.0   3rd Qu.:3615  
##  Max.   :46.60   Max.   :8.000   Max.   :455.0   Max.   :230.0   Max.   :5140  
##                                                                                
##   acceleration        year           origin                      name    
##  Min.   : 8.00   Min.   :70.00   Min.   :1.000   amc matador       :  5  
##  1st Qu.:13.78   1st Qu.:73.00   1st Qu.:1.000   ford pinto        :  5  
##  Median :15.50   Median :76.00   Median :1.000   toyota corolla    :  5  
##  Mean   :15.54   Mean   :75.98   Mean   :1.577   amc gremlin       :  4  
##  3rd Qu.:17.02   3rd Qu.:79.00   3rd Qu.:2.000   amc hornet        :  4  
##  Max.   :24.80   Max.   :82.00   Max.   :3.000   chevrolet chevette:  4  
##                                                  (Other)           :365  
```

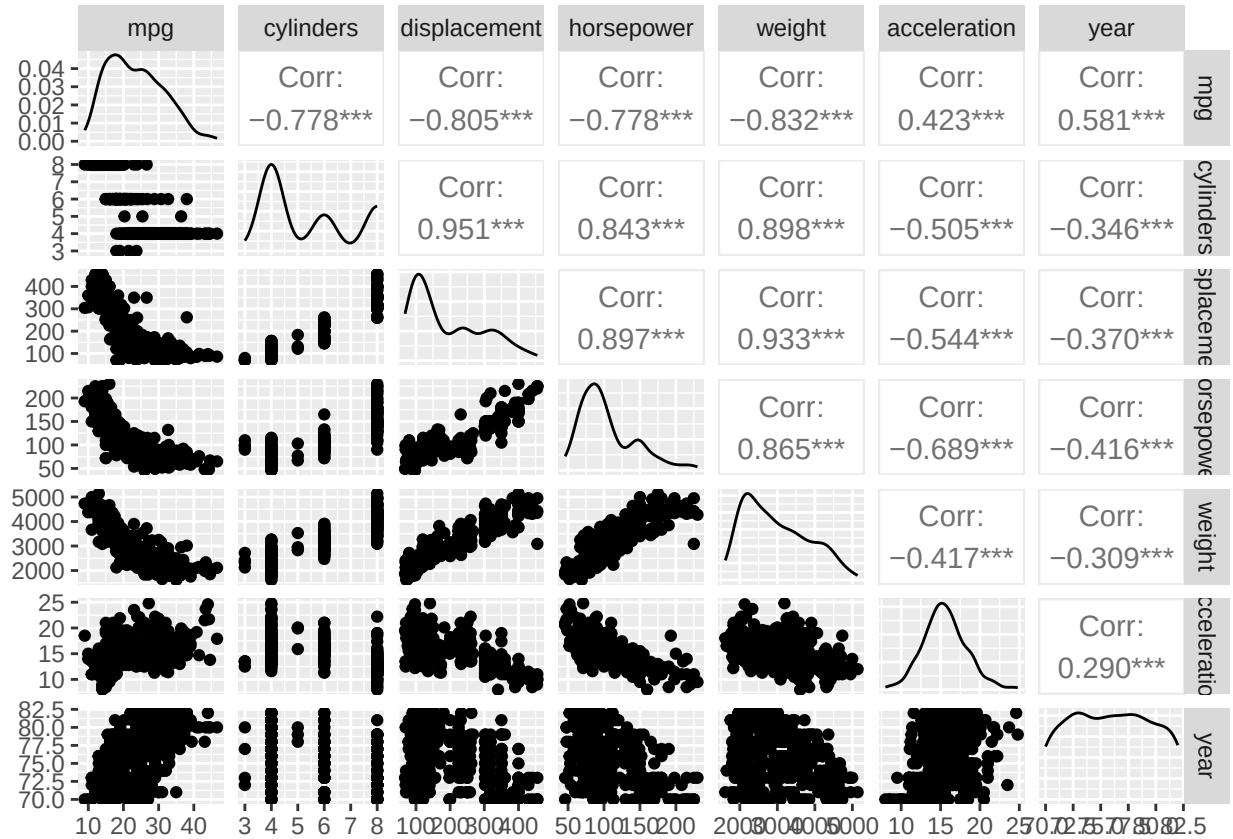
```r
# Check for missing values (should be none, but always good to confirm)
auto |> summarise(across(everything(), ~ sum(is.na(.))))
```

```
## # A tibble: 1 x 9
##     mpg cylinders displacement horsepower weight acceleration  year origin  name
##   <int>     <int>        <int>      <int>  <int>        <int> <int>  <int> <int>
## 1     0         0            0          0      0            0     0      0     0
```

```r
# Basic EDA
# Response distribution
auto |>
  ggplot(aes(x = acceleration)) +
  geom_histogram(bins = 30)
```




```r
# Quick pairs plot for a few numeric variables
auto_small <- auto |>
  select(-name, -origin)

GGally::ggpairs(auto_small)
```



Interpretation from paired plots

The ggpairs plot suggests several predictors are strongly related to each other (notably displacement, horsepower, and weight), which indicates potential multicollinearity in a multiple linear regression model. Acceleration shows visible relationships with these predictors (generally stronger than with mpg/year), so a linear model is reasonable as a starting point. Because some predictors move together, we should expect that individual p-values and coefficient signs/magnitudes may change as variables are removed during backward selection.

```r
# Train/test split
auto_split <- initial_split(auto_small, prop = 0.80, strata = acceleration)
auto_train <- training(auto_split)
auto_test  <- testing(auto_split)

# Model specification
lm_spec <- linear_reg() |>
  set_mode("regression") |>
  set_engine("lm")

lm_spec
```

```
## Linear Regression Model Specification (regression)
##
## Computational engine: lm
```

Initial Model

```r
# Initial model with all variables
belim_model <- lm_spec |>
  fit(acceleration ~ ., data = auto_train)

# Model output
model.metrics <- glance(belim_model)
model.metrics
```

```
## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1     0.627         0.620  1.69      85.5 2.21e-62     6  -602. 1220. 1250.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

```r
tidy(belim_model) |> arrange(p.value)|> mutate((remove=p.value > 0.05))
```

```
## # A tibble: 7 x 6
##   term         estimate std.error statistic  p.value `(remove = p.value > 0.05)`
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl> <lgl>
## 1 horsepower   -0.0925   0.00622    -14.9   5.15e-38 FALSE
## 2 weight        0.00386  0.000399     9.67  1.85e-19 FALSE
## 3 (Intercept)  19.7      2.42         8.11  1.21e-14 FALSE
## 4 displacement -0.0129   0.00435     -2.97  3.25e- 3 FALSE
## 5 year         -0.0587   0.0365      -1.61  1.08e- 1 TRUE
## 6 mpg           0.0357   0.0283       1.26  2.07e- 1 TRUE
## 7 cylinders     0.0236   0.191        0.124 9.02e- 1 TRUE
```

Remove cylinders

```r
belim_model <- lm_spec |>
  fit(acceleration ~ ., data = select(auto_train,-cylinders))

rbind(model.metrics,glance(belim_model)) -> model.metrics
tidy(belim_model) |>arrange(p.value) |> mutate(remove=p.value>0.05)
```

```
## # A tibble: 6 x 6
##   term         estimate std.error statistic  p.value remove
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl> <lgl>
## 1 horsepower   -0.0926   0.00620     -14.9  3.06e-38 FALSE
## 2 weight        0.00386  0.000398      9.69 1.48e-19 FALSE
## 3 (Intercept)  19.7      2.37          8.31 3.13e-15 FALSE
## 4 displacement -0.0125   0.00300      -4.17 3.89e- 5 FALSE
## 5 year         -0.0585   0.0364       -1.61 1.09e- 1 TRUE
## 6 mpg           0.0354   0.0281        1.26 2.09e- 1 TRUE
```

Remove mpg

```r
belim_model <- lm_spec |>
  fit(acceleration ~ ., data = select(auto_train,-cylinders, -mpg))

rbind(model.metrics,glance(belim_model)) -> model.metrics
tidy(belim_model) |>arrange(p.value) |> mutate(remove=p.value>0.05)
```

```
## # A tibble: 5 x 6
##   term         estimate std.error statistic  p.value remove
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl> <lgl>
## 1 horsepower   -0.0931   0.00619     -15.0  1.23e-38 FALSE
## 2 weight        0.00361  0.000346     10.4  4.99e-22 FALSE
## 3 (Intercept)  19.2      2.34          8.21 6.19e-15 FALSE
## 4 displacement -0.0122   0.00299      -4.08 5.64e- 5 FALSE
## 5 year         -0.0309   0.0291       -1.06 2.88e- 1 TRUE
```

Remove year

```
belim_model <- lm_spec |>
  fit(acceleration ~ ., data = select(auto_train,-cylinders, -mpg, -year))

rbind(model.metrics,glance(belim_model)) -> model.metrics
tidy(belim_model) |>arrange(p.value) |> mutate(remove=p.value>0.05)
```

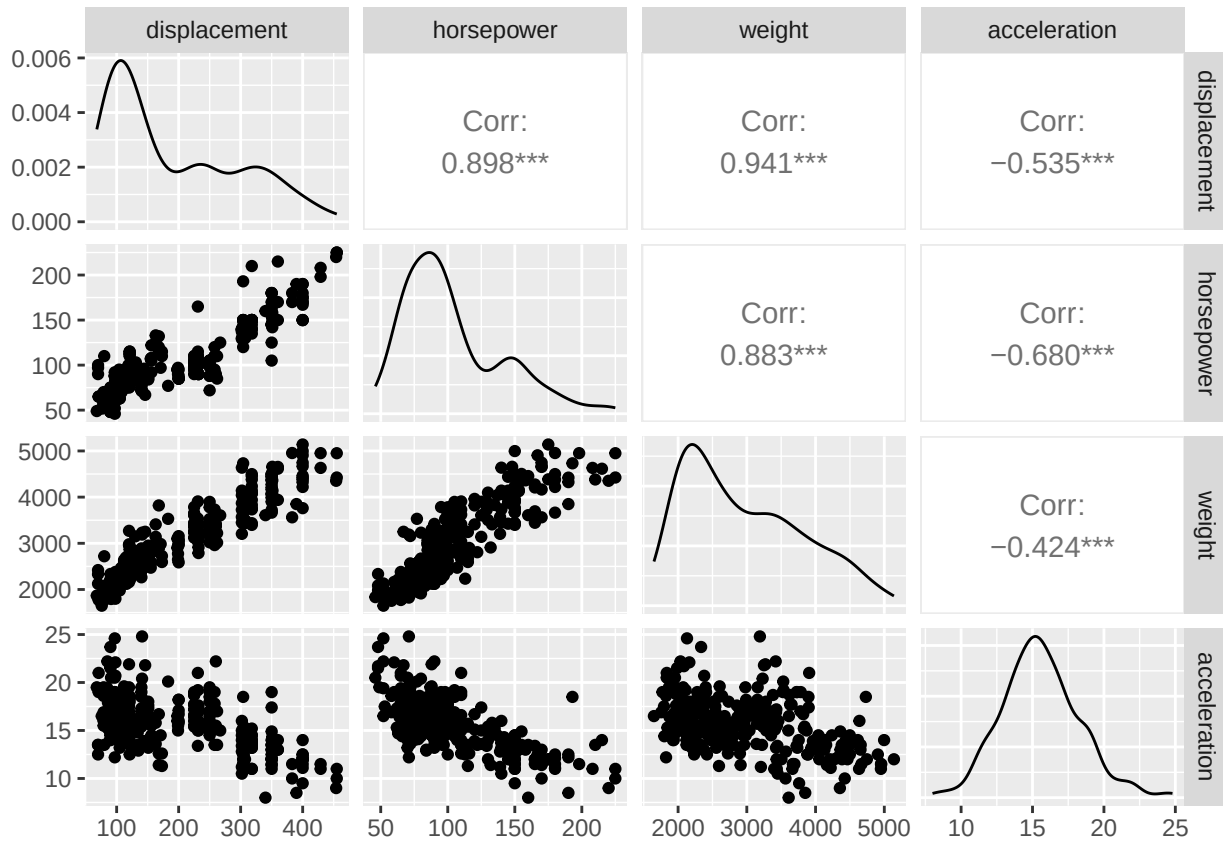
```
## # A tibble: 4 x 6
##   term         estimate std.error statistic  p.value remove
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl> <lgl>
## 1 (Intercept)  16.8      0.544        30.9  3.03e-96 FALSE
## 2 horsepower   -0.0915   0.00601     -15.2  2.02e-39 FALSE
## 3 weight        0.00356  0.000342     10.4  7.14e-22 FALSE
## 4 displacement -0.0119   0.00298      -4.01 7.71e- 5 FALSE
```

```
model.metrics
```

```
## # A tibble: 4 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1     0.627         0.620  1.69      85.5 2.21e-62     6  -602. 1220. 1250.
## 2     0.627         0.621  1.68     103.  2.09e-63     5  -602. 1218. 1244.
## 3     0.625         0.620  1.68     128.  3.79e-64     4  -603. 1218. 1240.
## 4     0.624         0.620  1.68     170.  4.69e-65     3  -603. 1217. 1235.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

Everything is significant now. This is our final model.

```
ggpairs(data = select(auto_train,-c(cylinders, mpg, year)) |>
          na.omit())
```



The plot shows strong evidence of multicollinearity among the predictors, between displacement, horsepower, and weight. The reported correlations between the predictors are extremely high: displacement–weight (r = 0.941), displacement–horsepower (r = 0.898), and horsepower–weight (r = 0.883). These nearly linear scatterplot patterns indicate that these variables carry overlapping information about vehicle "size/power." As a result, in a multiple linear regression, it becomes difficult to isolate each variable's unique contribution: coefficient estimates can become unstable, standard errors can be inflated, and variables may appear non-significant even if they are related to acceleration—because much of their explanatory power is shared with the other correlated predictors. This is why backward selection (or VIF-based screening/regularization) is helpful here: it reduces redundancy among predictors while keeping the model interpretable. But for this analysis, the variables are not removed because of their significance shown by the p-value.

```
vif(belim_model$fit)
```

```
## displacement   horsepower       weight
##    10.551274     5.521593     9.316368
```

Since VIF for displacement is greater than 10 (which is the threshold), removing displacement from the final model.

```
belim_model <- lm_spec |>
  fit(acceleration ~ ., data = select(auto_train,-cylinders, -mpg, -year, -displacement))

rbind(model.metrics,glance(belim_model)) -> model.metrics
tidy(belim_model) |>arrange(p.value) |> mutate(remove=p.value>0.05)
```

```
## # A tibble: 3 x 6
##   term        estimate std.error statistic   p.value remove
##   <chr>          <dbl>     <dbl>     <dbl>     <dbl> <lgl>
```

```
## 1 (Intercept) 18.4      0.359         51.4 1.94e-153 FALSE
## 2 horsepower  -0.102    0.00558      -18.2 8.56e- 51 FALSE
## 3 weight       0.00258  0.000245      10.5 2.65e- 22 FALSE
```

```r
vif(belim_model$fit)
```

```
## horsepower     weight
##   4.547252   4.547252
```

Removing `displacement` improved the VIF score, but let's also check the model performance.

```r
model.metrics
```

```
## # A tibble: 5 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1     0.627         0.620  1.69      85.5 2.21e-62     6  -602. 1220. 1250.
## 2     0.627         0.621  1.68     103.  2.09e-63     5  -602. 1218. 1244.
## 3     0.625         0.620  1.68     128.  3.79e-64     4  -603. 1218. 1240.
## 4     0.624         0.620  1.68     170.  4.69e-65     3  -603. 1217. 1235.
## 5     0.604         0.602  1.72     236.  6.66e-63     2  -611. 1231. 1246.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

Since, the model performance did not improve, rather degraded (looking at R^2 values), I am deciding not to remove `displacement` from the final model.

Final Model

$$\text{acceleration} \sim \text{displacement} + \text{horsepower} + \text{weight}$$

```r
belim_model <- lm_spec |>
  fit(acceleration ~ ., data = select(auto_train, -cylinders, -mpg, -year))
glance(belim_model)
```

```
## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1     0.624         0.620  1.68      170. 4.69e-65     3  -603. 1217. 1235.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

```r
# Coefficient table for interpretation
tidy(belim_model)
```

```
## # A tibble: 4 x 5
##   term         estimate std.error statistic  p.value
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)  16.8      0.544        30.9  3.03e-96
## 2 displacement -0.0119   0.00298      -4.01 7.71e- 5
## 3 horsepower   -0.0915   0.00601     -15.2  2.02e-39
## 4 weight        0.00356  0.000342     10.4  7.14e-22
```

Interpreting the fitted multiple linear regression model

$$\widehat{acceleration} = 16.7808 - 0.0119 \times displacement - 0.0915 \times horssepower + 0.0036 \times weight$$

Interpretation of all predictors

- **Intercept = 16.7808**

The intercept (16.7808) is the predicted 0–60 acceleration time (seconds) when displacement, horsepower, and weight are all zero. Since a car cannot realistically have zero displacement/horsepower/weight, the intercept is mainly a baseline mathematical reference point rather than a meaningful real-world value.

Displacement

- **Displacement = -0.0119**

Holding horsepower and weight constant, a one-unit increase in displacement is associated with a 0.0119 second decrease in predicted 0–60 time. In other words, larger displacement is associated with slightly faster acceleration (smaller time), although the effect per single cubic inch is small.

Horsepower

- **Horsepower = -0.0915**

Holding displacement and weight constant, a one-unit increase in horsepower is associated with a 0.0915 second decrease in predicted 0–60 time. That means higher horsepower tends to produce faster acceleration (lower time), and the size of the effect is noticeable compared with displacement.

Weight

- **Weight = 0.0036**

Holding displacement and horsepower constant, a one-unit increase in weight is associated with a 0.0036 second increase in predicted 0–60 time. That means heavier cars are predicted to accelerate more slowly (larger time). Because weight is measured in pounds, changes of hundreds of pounds can translate into a meaningful change in acceleration time.

Accuracy of the model

```r
# Predictions + accuracy
auto_metrics <- metric_set(rmse, rsq, mae)

train_pred <- predict(belim_model, new_data = auto_train) |>
  bind_cols(auto_train)

test_pred <- predict(belim_model, new_data = auto_test) |>
  bind_cols(auto_test)

# Metrics
auto_metrics(train_pred, truth = acceleration, estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard       1.67
## 2 rsq     standard       0.624
## 3 mae     standard       1.27
```

```r
auto_metrics(test_pred, truth = acceleration, estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard       1.89
```

```
## 2 rsq     standard       0.629
## 3 mae     standard       1.41
```

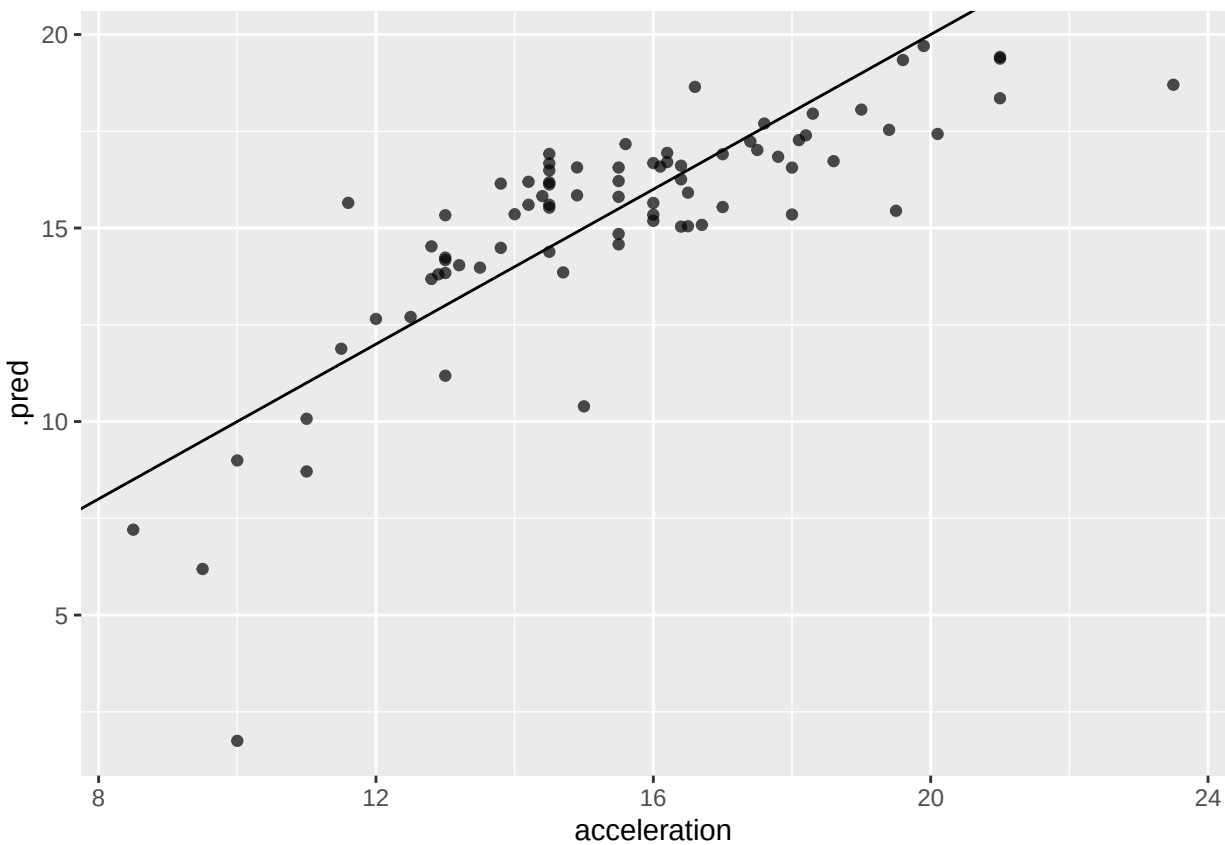
```r
rsq(train_pred, truth = acceleration, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard       0.624
```

```r
rsq(test_pred, truth = acceleration, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard       0.629
```

```r
# Quick plot: predicted vs actual on test set
test_pred |>
  ggplot(aes(x = acceleration, y = .pred)) +
  geom_point(alpha = 0.7) +
  geom_abline(slope = 1, intercept = 0)
```



Interpretation The predicted vs. actual plot on the test set shows predictions mostly tracking the 45° line, meaning the model captures the main trend in acceleration reasonably well. The test RMSE is 1.89 and MAE is 1.41, so typical prediction errors are on the order of 1–2 seconds, which matches the visible scatter around the line. There is a mild pattern where the model tends to underpredict at the highest acceleration times (some points at high actual values fall below the line), suggesting the linear model struggles a bit at the extreme slow-acceleration end. Overall, test $R^2 = 0.629$, so the model explains about

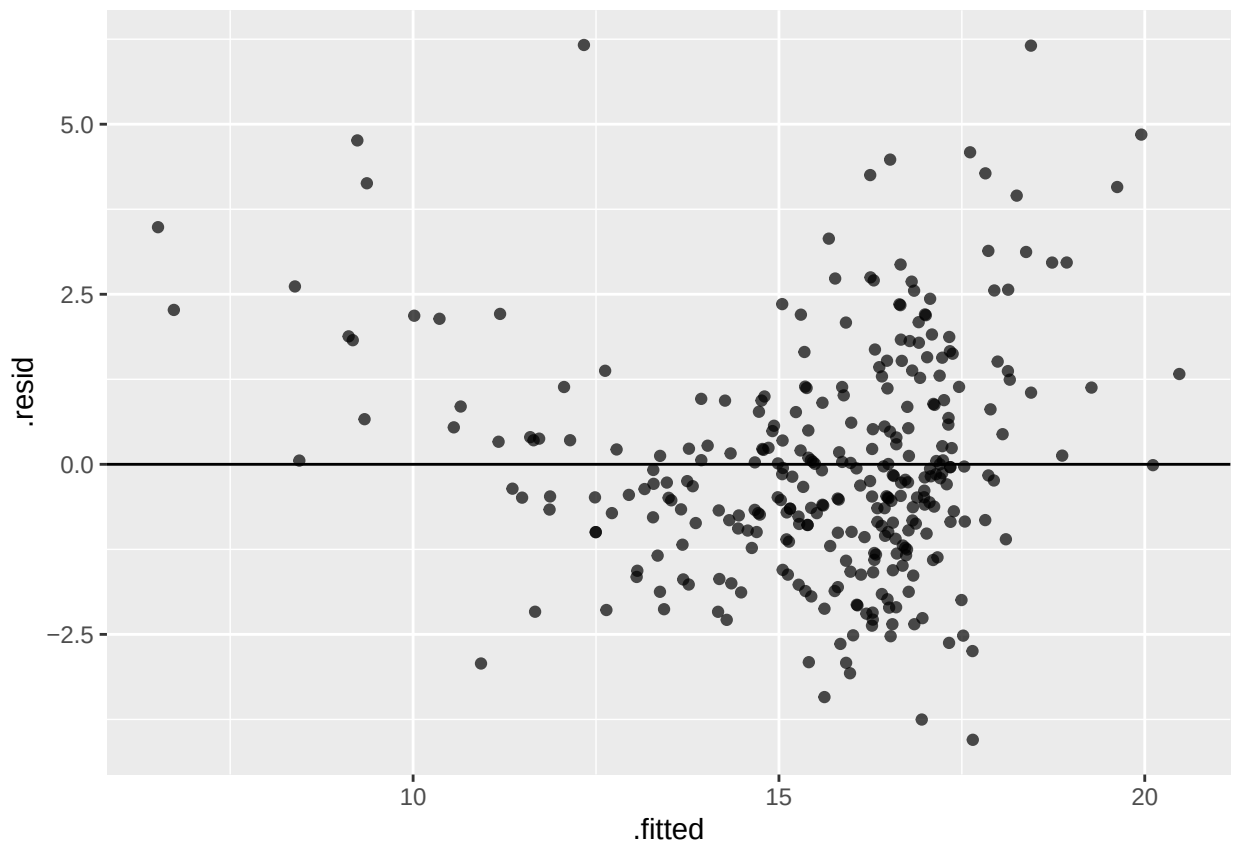
63% of the variation in acceleration on unseen data.

Model Assessment

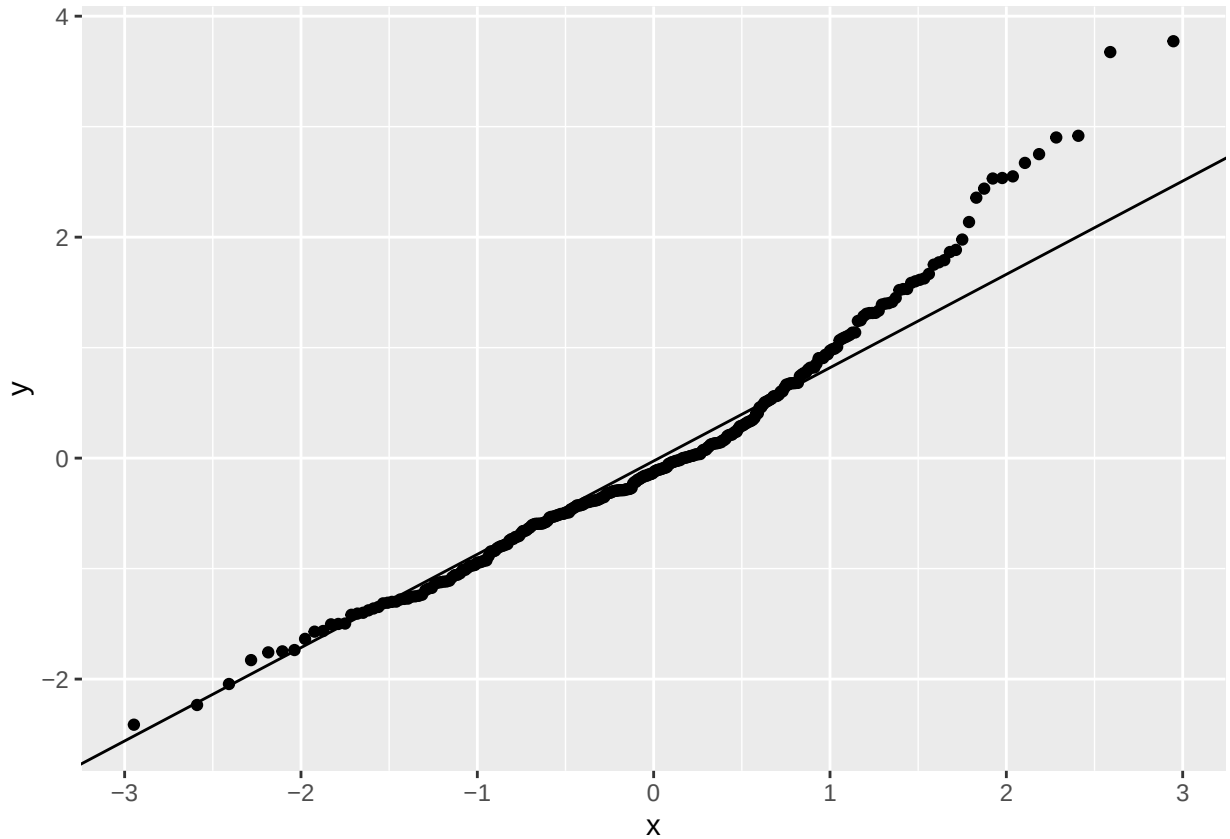
```r
# Diagnostics / model assessment
lm_engine <- extract_fit_engine(belim_model)

# Get fitted values + residuals in a tidy way
diag_df <- broom::augment(lm_engine)

# Confidence Interval for coefficients
broom::tidy(lm_engine, conf.int = TRUE, conf.level = 0.95)
```

```
## # A tibble: 4 x 7
##   term         estimate std.error statistic  p.value conf.low conf.high
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
## 1 (Intercept)  16.8      0.544        30.9  3.03e-96 15.7      17.9
## 2 displacement -0.0119   0.00298      -4.01 7.71e- 5 -0.0178   -0.00608
## 3 horsepower   -0.0915   0.00601     -15.2  2.02e-39 -0.103    -0.0796
## 4 weight        0.00356  0.000342     10.4  7.14e-22  0.00288   0.00423
```

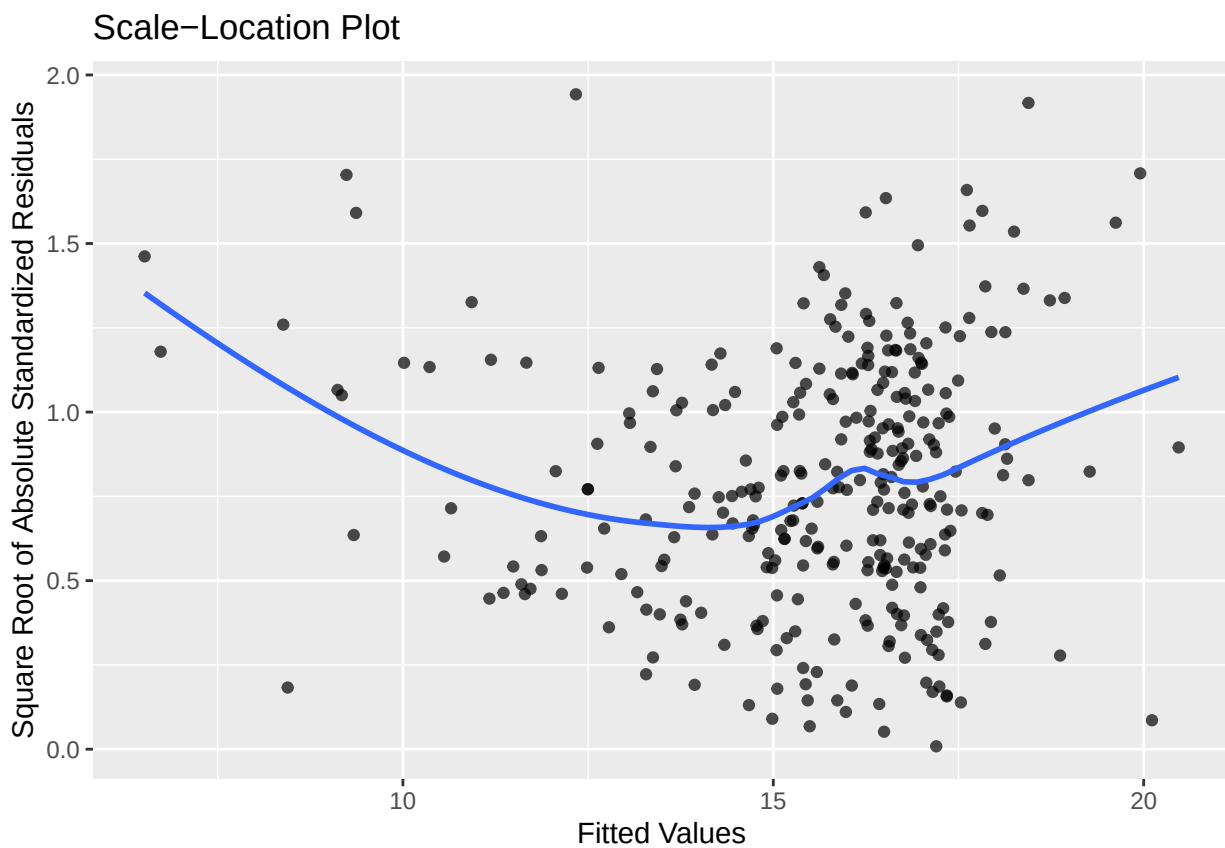
```r
# Residuals vs fitted (linearity + constant variance check)
diag_df |>
  ggplot(aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.7) +
  geom_hline(yintercept = 0)
```



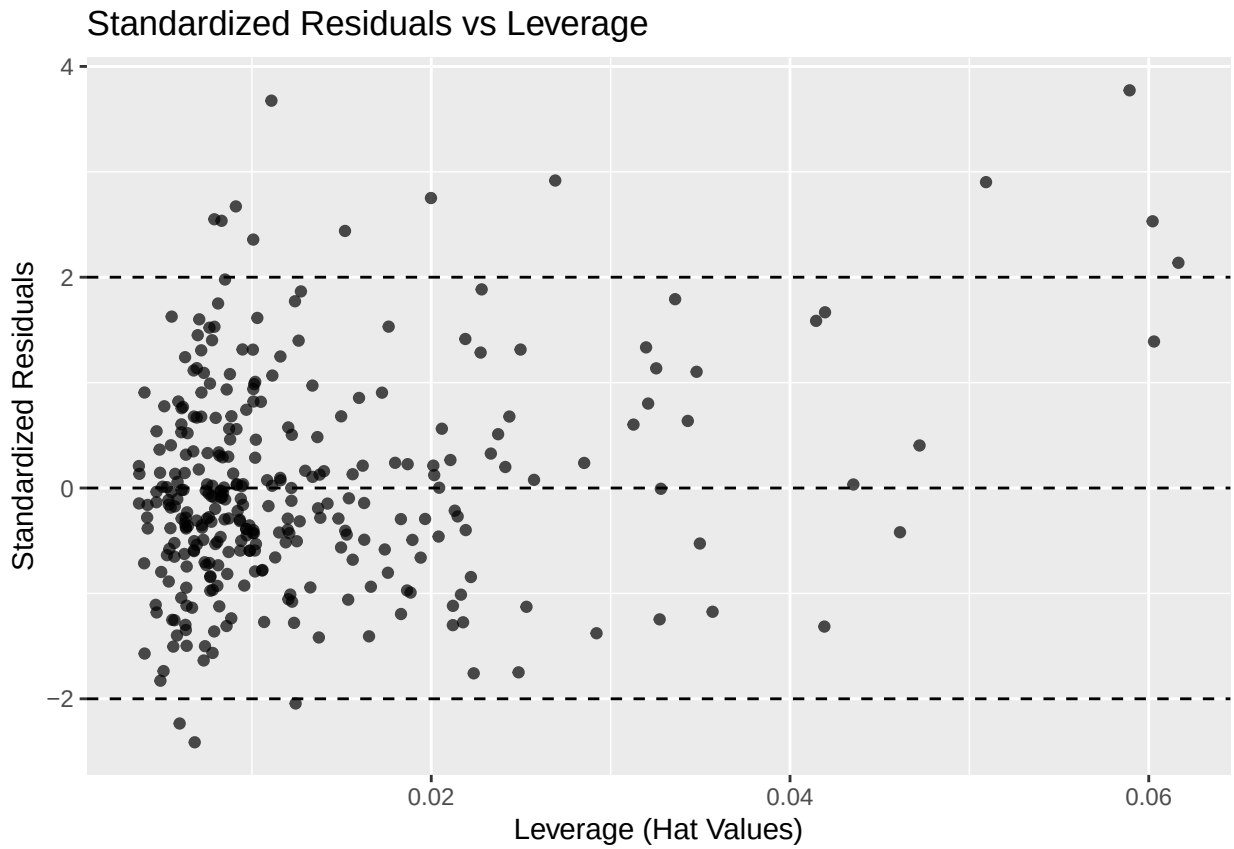
```r
# Normal Q-Q plot (normality of residuals)
diag_df |>
  ggplot(aes(sample = .std.resid)) +
  stat_qq() +
  stat_qq_line()
```




```r
# Scale-Location plot (another look at constant variance)
diag_df |>
  ggplot(aes(x = .fitted, y = sqrt(abs(.std.resid)))) +
  geom_point(alpha = 0.7) +
  geom_smooth(method = "loess", se = FALSE) +
  labs(
    title = "Scale-Location Plot",
    x = "Fitted Values",
    y = "Square Root of Absolute Standardized Residuals"
  )
```




```r
# Leverage / influence
diag_df |>
  ggplot(aes(x = .hat, y = .std.resid)) +
  geom_point(alpha = 0.7) +
  geom_hline(yintercept = c(-2, 0, 2), linetype = "dashed") +
  labs(
    title = "Standardized Residuals vs Leverage",
    x = "Leverage (Hat Values)",
    y = "Standardized Residuals"
  )
```



The diagnostics suggest the linear model is broadly reasonable but not perfect.

The residuals vs fitted plot shows residuals generally centered around zero without strong curvature, so there isn't clear evidence of a major non-linear structure being missed. However, assumptions are slightly imperfect because variance looks a bit uneven and there are a few outliers.

The Q–Q plot is close to linear in the middle but bends away in the tails, indicating mild non-normality / heavier tails, driven by a few larger standardized residuals.

The scale–location plot shows the LOESS curve dipping and then rising again, which suggests slightly non-constant variance (residual spread changes with fitted values rather than staying perfectly flat).

Finally, the standardized residuals vs leverage plot shows most observations have low leverage, but there are a few higher-leverage points (around hat values near ~0.06), including one with a large positive standardized residual; these may be potentially influential and worth checking, though they appear to be only a small number of cases.